

TDDE01 lab1 report

Oskar Andersson, Ahmad Soltani, Zurab Gergaia

November 2025

1 Statement of Contribution

- Ahmad Soltani did all work for part 1, Digit Recognition with k-Nearest Neighbors, and answered question 4.1.
- Zurab Gergaia did all work for part 2, Linear Regression and ridge regression, and answered question 4.2.
- Oskar Andersson did all work for part 3, Logistic Regression and Basis Function Expansion, and answered question 4.3.

2 Digit Recognition with K-Nearest Neighbors

This assignment focuses on recognizing digit numbers by training models using the R inbuilt function `knn()`. The dataset for this task is provided as a CSV file, which consists of 3,823 rows of digit data. Each row contains 64 features representing an 8×8 handwritten digit image, and the last column serves as the label (digits 0–9).

2.1 Data Partition

The data is divided into three partitions: 50% for training, 25% for testing, and 25% for validation. The function `sample()` is used to randomly distribute the data samples. Additionally, `set.seed(12345)` ensures reproducibility so that the same random values are generated each time the code runs.

2.2 Estimation of Confusion Matrices and Misclassification Errors

To estimate confusion matrices and misclassification errors, we use the inbuilt function `kknn()` with $K = 30$ and the kernel set to “rectangular” to train the models. Two models are trained:

1. Model 1: Trained and tested on the same dataset (training set).
2. Model 2: Trained on the training set and tested on the test set.

2.2.1 Confusion Matrices

A confusion matrix is a table that evaluates the performance of the trained model. The rows represent true labels, columns represent predicted labels, and the diagonal shows correct classifications. For example, in the training dataset, digit “8” was correctly classified 188 times, misclassified as “1” 10 times, as “6” 2 times, and as “9” 2 times (see Figure 1).

		Pred									
True		0	1	2	3	4	5	6	7	8	9
		0	202	0	0	0	0	0	0	0	0
1	0	179	11	0	0	0	0	0	1	1	3
2	0	1	190	0	0	0	0	0	1	0	0
3	0	0	0	185	0	1	0	1	0	1	1
4	1	3	0	0	159	0	0	7	1	4	
5	0	0	0	1	0	171	0	1	0	8	
6	0	2	0	0	0	0	190	0	0	0	
7	0	3	0	0	0	0	0	178	1	0	
8	0	10	0	2	0	0	2	0	188	2	
9	1	3	0	5	2	0	0	3	3	183	

(a) training

		Pred									
True		0	1	2	3	4	5	6	7	8	9
		0	82	0	0	0	1	0	1	0	0
1	0	90	2	0	0	0	0	0	0	0	3
2	0	1	92	0	0	0	0	0	1	1	1
3	0	0	0	85	0	2	0	3	1	1	
4	0	1	0	0	89	0	1	6	0	5	
5	0	1	0	1	0	97	1	1	0	7	
6	0	0	0	0	0	0	97	0	0	0	
7	0	1	0	1	0	0	0	99	0	0	
8	0	7	0	0	0	0	0	0	84	0	
9	0	2	0	0	0	0	0	2	1	86	

(b) test

Figure 1: Confusion matrices for predicted vs true digits

2.2.2 Misclassification Error

The 30-nearest neighbor classifier achieved:

Training error: **0.04500262**, Test error: **0.0585162**.

This corresponds to approximately 96% accuracy on training data and 95% accuracy on test data. The slightly higher accuracy on training data is expected. Overall, these low error rates indicate that the model performs well.

2.3 Classifying the Easiest and Hardest Cases of Digit “8”

In this part, we identify:

- Two easiest cases where digit “8” is classified with the highest confidence.
- Three hardest cases where digit “8” is classified with the lowest confidence.



Figure 2: Three hardest cases



Figure 3: Two easiest cases

We use the function `heatmap()` to visualize these cases. The easiest cases have a more clear and typical shapes the digit “8”, while the hardest cases are visually ambiguous. Figure 2 shows the hardest cases, and Figure 3 shows the easiest cases.

2.4 Misclassification Error for Different K

We validate and compare misclassification errors for $K = 1$ to $K = 30$ on training and validation datasets. As shown in Figure 3, as K increases, model complexity changes, affecting error rates. For training data, $K = 1$ and $K = 2$ yield zero error, while for validation data, the optimal K is 3 with an error of approximately **0.018**.

Optimal $K = 3$: Training error **0.0115123**, Test error **0.03134796**.

Optimal $K = 2$: Training error **0**, Test error **0.0292581**.

These results show that while training error can reach zero, validation and test errors remain non-zero, which is expected for unseen data (see Figure 4).

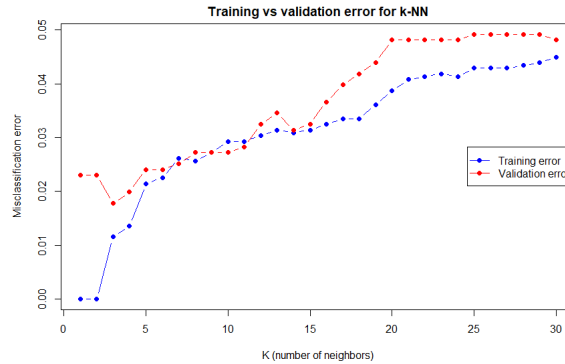


Figure 4: The Y-axis represents the misclassification error, while the X-axis represents K values ranging from 1 to 30 for training (blue) and validation (red).

2.5 Cross-Entropy Error for Different K

We compute cross-entropy errors for $K = 1$ to $K = 30$ on the validation dataset. Figure 5 shows that cross-entropy is highest at $K = 1$ and optimal at $K = 5$.

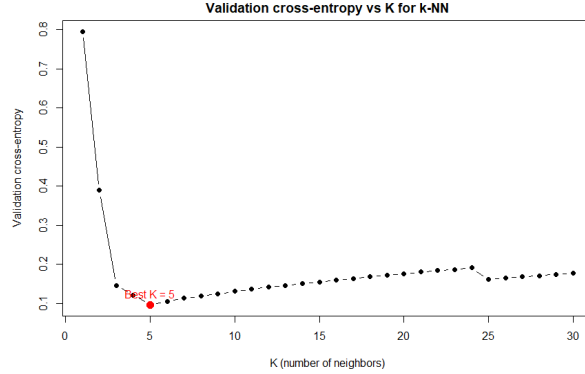


Figure 5: The Y-axis represents cross-entropy, and the X-axis represents K values ranging from 1 to 30.

Since labels are multi-class (0–9) and drawn from a multinomial distribution, cross-entropy evaluates how much probability the model assigns to the true class. It uses the full probability vector, distinguishing models that classify correctly but differ in confidence calibration.

3 Linear regression and ridge regression

3.1 Preparations before training

According to the Lab, a random seed has been set to "12345" and training data was split in a 60/40 fashion, where 60% of data was used for training and the remaining 40% was used for testing trained model. Furthermore, using a scaler from the **caret** library, both training and testing sets have been scaled, in order to make every feature equally important, and increase convergence speed.

3.2 Computing linear regression model

To fit a linear model, a linear formula must be devised that defines characteristic features, which contribute to the final prediction. According to the lab, the formula looks like this:

```
motor_UPDRS ~ 0 +
  Jitter... + Jitter.Abs. + Jitter.RAP + Jitter.DDP +
  Jitter.PPQ5 + Shimmer + Shimmer.dB. + Shimmer.APQ3 +
  Shimmer.APQ5 + Shimmer.APQ11 + Shimmer.DDA +
  NHR + HNR + RPDE + DFA + PPE
```

Because the data have been scaled and every feature is equally important, we set the intercept to 0 in the aforementioned formula. Figure 1, shows the summary of trained model using `lm()` function in R. We can see, that **Jitter.Abs.**, **Shimmer**, **Shimmer.APQ5**, **Shimmer.APQ11**, **NHR**, **HNR**, **DFA**, **PPE** make a significant contribution to the model, because their p-values are very close to 0. Training MSE is calculated by:

1. Taking trained model's residuals
2. Squaring them
3. Finding the mean, which in this case is **0.878543102826272**

To find test MSE:

1. Predictions are made on the test data explicitly
2. Errors are defined by subtracting a real result from predictions
3. Errors are squared and the mean is calculated, which is **0.935447712156729**

Coefficients:				
	Estimate	Std. Error	t value	Pr(> t)
Jitter...	0.186931	0.149561	1.250	0.211431
Jitter.Abs.	-0.169609	0.040805	-4.157	3.31e-05 ***
Jitter.RAP	-5.269544	18.834160	-0.280	0.779658
Jitter.DDP	5.249558	18.837525	0.279	0.780510
Jitter.PPQ5	-0.074568	0.087766	-0.850	0.395592
Shimmer	0.592436	0.205981	2.876	0.004050 **
Shimmer.dB.	-0.172655	0.139316	-1.239	0.215315
Shimmer.APQ3	32.070932	77.159242	0.416	0.677694
Shimmer.APQ5	-0.387507	0.113789	-3.405	0.000668 ***
Shimmer.APQ11	0.305546	0.061236	4.990	6.34e-07 ***
Shimmer.DDA	-32.387241	77.158814	-0.420	0.674695
NHR	-0.185387	0.045567	-4.068	4.84e-05 ***
HNR	-0.238543	0.036395	-6.554	6.41e-11 ***
RPDE	0.004068	0.022664	0.179	0.857556
DFA	-0.280318	0.020136	-13.921	< 2e-16 ***
PPE	0.226467	0.032881	6.887	6.70e-12 ***

Figure 6: Training summary

3.3 Log-likelihood

We can reason about model's accuracy by utilizing a probabilistic framework and computing a log-likelihood for a given model and training set. In fact, log-likelihood serves as a basic building block for most loss functions in machine learning. Before we calculate it, we assume that errors are normally distributed with a mean of 0. Moreover, the calculation also requires a dispersion σ . We can either assume σ is a constant or we can learn it. Calculations involve:

1. Making predictions on a training data and calculating errors
2. Finding probabilities of all errors
3. Summing all probabilities up

3.4 Ridge

To guard against overfitting, we can add a penalty to the standard linear model. This is achieved by shrinking the model's coefficients and controlling the penalty by a parameter λ . As λ increases, the model is penalized more heavily for having large coefficients. It is done by:

1. Calculating a negative log-likelihood
2. Multiplying λ by a sum of a models coefficients' squared
3. Adding aforementioned calculations up

3.5 RidgeOpt

To create a wrapper around a generic function **optim()** that computes optimal θ and σ , we need to create a new cost function. This function receives 1 parameter that combines θ and σ into 1 vector and returns back the penalty calculated by **Ridge** function. To make **optim()** try only positive values for σ , the natural logarithm of σ is calculated and inside a cost function, the exponent of σ is found.

3.6 Degree of freedom

In ridge regression, because the parameters are shrunk, the degree of freedom is not simply counting the number of features in the model. Instead, it is a continuous value. To calculate it, we can utilize the following matrix formula:

$$df(\lambda) = tr(X(X^T X + \lambda I)^{-1} X^T)$$

Where:

- **X** is the input matrix
- λ is ridge penalty parameter
- **I** is the identity matrix (matrix with ones on the main diagonal and zeros everywhere else)
- **tr** denotes the sum of diagonal elements

3.7 Computing RidgeOpt and DF

As table 1 demonstrates, for $\lambda(10)$, the training MSE worsens by **0.01** and the testing MSE improves by **0.001**. Unfortunately, this trade is not worth it, because training MSE takes much larger penalty, while test MSE is improved by small margin. Higher λ values only worsen the situation for both MSE(s). Hence, the most appropriate λ value seems to be **1**.

	Training MSE	Test MSE
$\lambda(1)$	0.889939187563786	0.938139983158269
$\lambda(100)$	0.89440582425537	0.937848061046732
$\lambda(1000)$	0.933419375762343	0.960844860375883

Table 1: Training and Testing MSE(s) for various λ

Table 2 shows, that as λ increases, degree of freedom is getting worse. Consequently the optimal value for λ is **1**.

	DF
$\lambda(1)$	13.8607362829965
$\lambda(100)$	9.92488712829543
$\lambda(1000)$	5.6439254878463

Table 2: Degrees of freedom for different λ s

4 Logistic Regression and Basis Function Expansion

4.1 Is diabetes easy to classify with only age and glucose level using standard logistic regression?

It should be roughly suitable for it. While there is some overlap between the two groups, most non-diabetic individuals are concentrated in the area encapsulated by $\hat{32}$ years of age and having a blood glucose level of between $\hat{60}$ to 140. It is possible to draw a gentle line roughly separating the two groups, which should make it suitable for logistic regression.

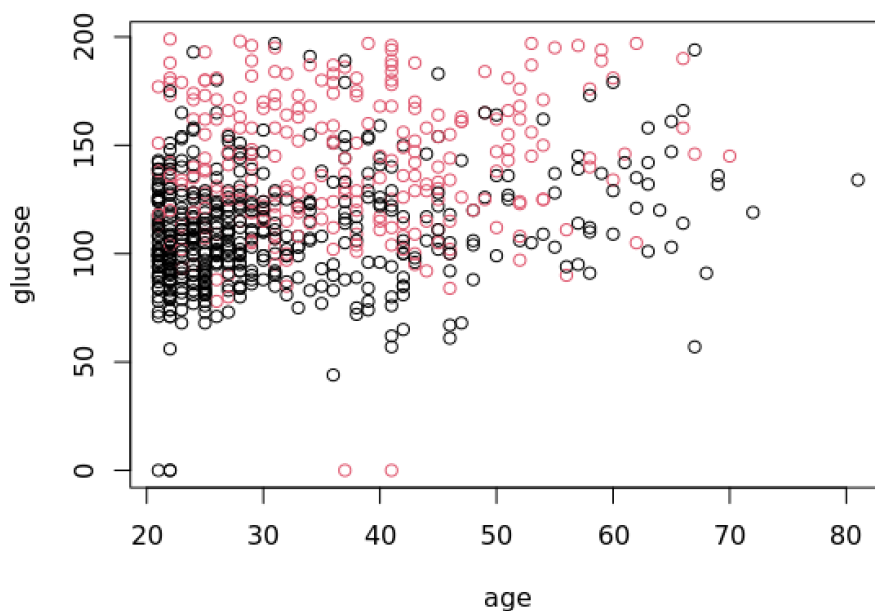


Figure 7: Plasma glucose concentration on age, colored by diabetes

4.2 Train a logistic regression model

We train a logistic regression model using R's `glm` (generalized linear model) function. Our parameters are:

- $y = \text{Diabetes}$
- $x_1 = \text{Age}$
- $x_2 = \text{Glucose}$
- $r = 0.5$

The probabilistic equation for this trained model is:

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \text{ glucose} + \beta_2 \text{ age})}} = \frac{1}{1 + e^{-(-5.90 + 0.06 \text{ glucose} + 0.02 \text{ age})}}.$$

And the misclassification rate is 0.314.

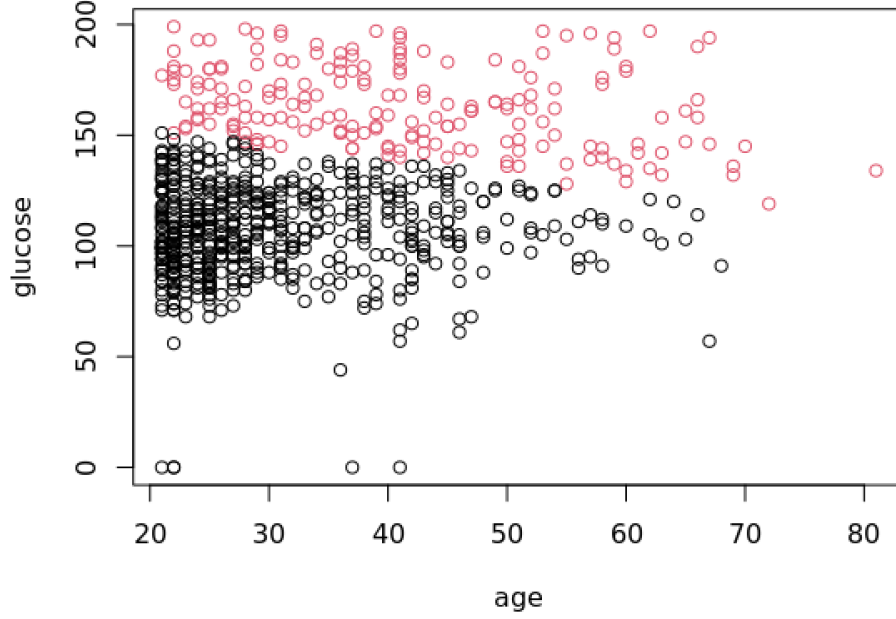


Figure 8: Logical regression model trained on glucose, age and diabetes level

This is not a very good model, as it misclassifies over 30% of predictions. As there are no exponents in the probabilistic equation, it is linear and as such the model has simply chosen the best line it could find, which happened to be this one. It would require some kind of nonlinearity to properly handle the noise in the dataset.

4.3 Decision boundary

The decision boundary is calculated by taking the probabilistic function and equating it to the threshold value

$$\frac{1}{1 + e^{-(\beta_0 + \beta_1 \text{ glucose} + \beta_2 \text{ age})}} = 0.5$$

We then solve for a given value, in this case glucose as it is along the y-axis.

$$\text{glucose} = \frac{-\beta_0 - \beta_2 \text{ age}}{\beta_1} = 98.33 - 0.33 \text{ age}.$$

We then take the maximum and minimum values of age and sample 10 points between them. Thus, we have a straight line equation where Y is the Glucose and X is Age. This is the model's decision. All observations above it will have a probability of 0.5 or higher and will be deemed positive, and all below will be deemed negative.

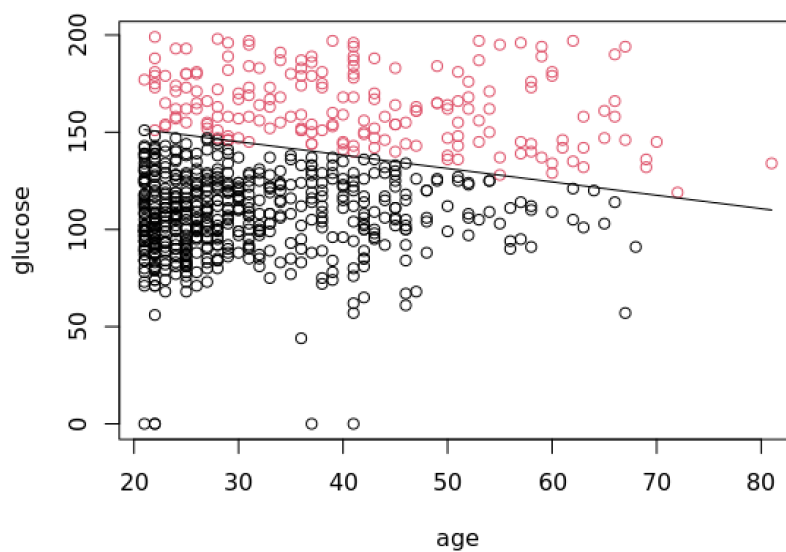


Figure 9: Decision Boundary

4.4 Different thresholds

Changing the threshold (the r -value) changes what probability we require for an observation to be classified as positive.

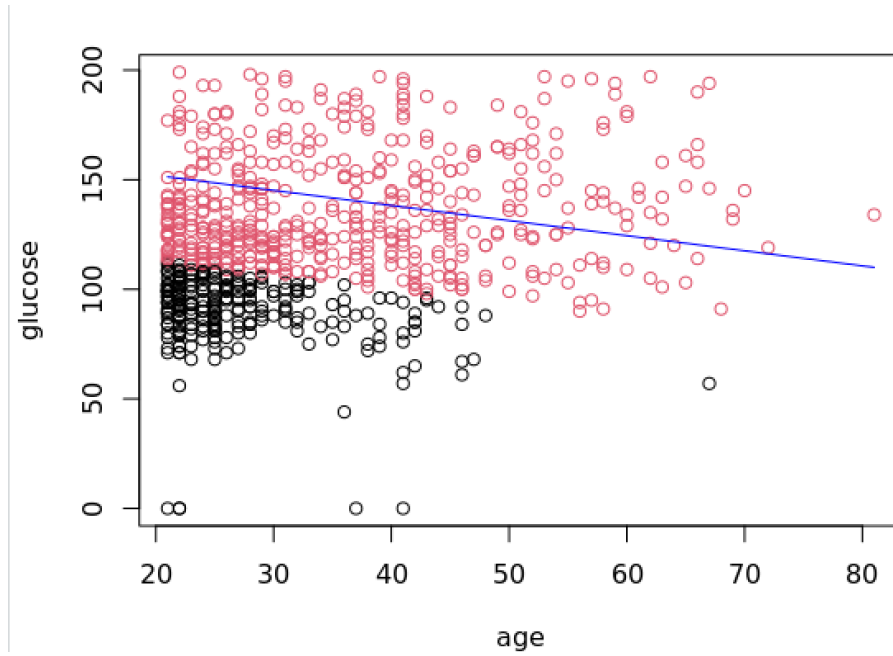


Figure 10: Threshold of 0.2. Blue line is the decision boundary a for threshold of 0.5

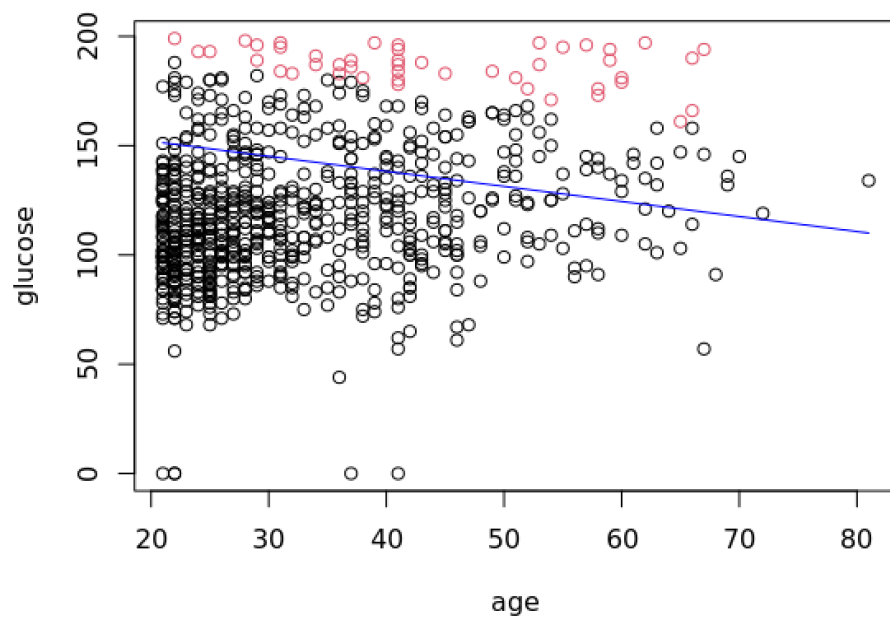


Figure 11: Threshold of 0.8. Blue line is the decision boundary for a threshold of 0.5

4.5 Basis function expansion

Using basis function expansion, we can introduce non-linearity into the model. Here, we can see that the decision boundary is not a straight line. The misclassification rate is 0.256, which is lower than the earlier 0.314 but still not particularly great as 1/4 of predictions are still wrong.

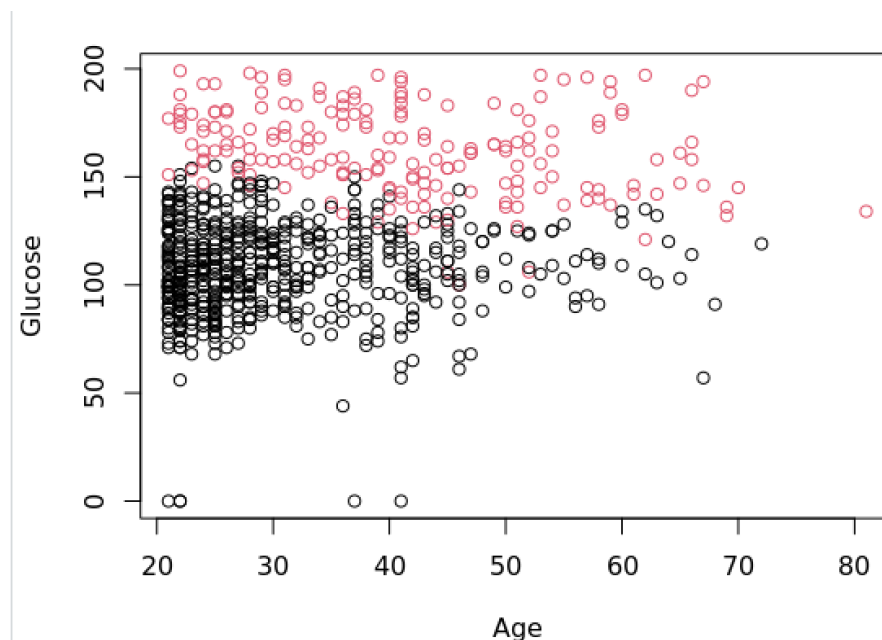


Figure 12: Basis Expansion

5 Theory

5.1 Why can it be important to consider various probability thresholds in the classification problems, according to the book?

According to the course book (page 50), many classification problems are asymmetric, meaning that correctly predicting some classes is more important than others. In such cases, it can be useful to use decision thresholds different from the standard choice $r=0.5$, which only minimises the total misclassification rate on average. But minimising the misclassification rate is not always the main goal, especially when the classes are imbalanced or the cost of different types of errors is very different. By considering various probability thresholds, we can adjust the classifier so that it better reflects the importance of different classes and errors in the specific application.

5.2 What ways of collecting correct values of the target variable for the supervised learning problems are mentioned in the book?

According to the book (page 14), there are two primary ways of collecting the correct values for supervised learning. **Jointly recording** x and y , where input and output are observed and recorded together. The other way involves **domain experts**, who manually label the output y according to observations made on the input data x .

5.3 How can one express the cost function of the linear regression in the matrix form, according to the book?

Equation 3.7 on page 40 of the book states that a linear regression model can be described in the form

$$y = X\theta + \epsilon,$$

where X and Y are matrices containing inputs and outputs x_i y_i $1 < i < n$ where n is the amount of observations in the training data, θ is a matrix of the values which we want to learn and ϵ is a matrix holding noise and/or errors. The goal is to find $\hat{y}$, which holds all the predicted outcomes depending on the unknown values in θ . A well trained model will have $y - \hat{y} = \epsilon$ be as low as possible, which means that $\hat{y}$ should be as similar to y as possible.

A Assignment 1 code

```
data <- read.csv("data/optdigits.csv", header = FALSE)

p <- ncol(data)

data[[p]] <- as.factor(data[[p]])
names(data)[p] <- "digit"

set.seed(12345)

n <- nrow(data)

id_train <- sample(1:n, size = floor(0.5 * n))
train <- data[id_train, ]
id_remaining <- setdiff(1:n, id_train)
id_valid <- sample(id_remaining, size = floor(0.25 * n))
valid <- data[id_valid, ]
id_test <- setdiff(id_remaining, id_valid)
test <- data[id_test, ]
```

```

library(kknn)

f_knn <- digit ~ .

knn_train <- kknn(
  formula = f_knn,
  train = train, #used to train the model,
  test = train, # how well the model learns.
  k = 30,
  kernel = "rectangular"
)

pred_train <- fitted(knn_train)

knn_test <- kknn(
  formula = f_knn,
  train = train,
  test = test,
  k = 30,
  kernel = "rectangular"
)

pred_test <- fitted(knn_test)

cm_train <- table(True = train$digit, Pred = pred_train)
cm_test  <- table(True = test$digit,  Pred = pred_test)

cat("\nConfusion matrix - TRAIN:\n")
print(cm_train)

cat("\nConfusion matrix - TEST:\n")
print(cm_test)

train_error <- mean(pred_train != train$digit)
test_error  <- mean(pred_test  != test$digit)

cat("\nTraining misclassification error:", train_error, "\n")
cat("Test misclassification error:",      test_error,  "\n")

#-----#Q3-----
prob_train <- knn_train$prob

idx_8 <- which(train$digit == "8")
prob_8 <- prob_train[idx_8, "8"]
rank_highest <- order(prob_8, decreasing = TRUE)

```

```

easiest_idx <- idx_8[rank_highest[1:2]]

rank_lowest <- order(prob_8, decreasing = FALSE)
hardest_idx <- idx_8[rank_lowest[1:3]]

cat("the easiest 8's:\n")
print(easiest_idx)
cat("their probabilities P(class=8):\n")
print(prob_8[rank_highest[1:2]])

cat("\nRow in TRAIN of the hardest 8's:\n")
print(hardest_idx)
cat("their prob P(class=8):\n")
print(prob_8[rank_lowest[1:3]])

#-----two easiest-----
plot_heatmap <- function(x, main = ""){
  m <- matrix(as.numeric(x), nrow = 8, ncol = 8, byrow = TRUE)

  heatmap(
    m,
    Rowv = NA, Colv = NA,
    scale = "none",
    col = gray.colors(256),
    main = main,
    labRow = NA, labCol = NA
  )
}

par(mfrow = c(1, 2))
feature_cols <- 1:(p - 1)

for (i in 1:2){
  row_idx <- easiest_idx[i]
  x <- train[row_idx, feature_cols]
  prob_val <- prob_8[rank_highest[i]]

  plot_heatmap(
    x,
    main = paste("easiest 8-#", i,
                 "\ntrain row", row_idx,
                 "\nP(class=8)=", round(prob_val, 3))
  )
}

```

```

#-----three hardest-----
par(mfrow = c(1, 3))

for (i in 1:3){
  row_idx <- hardest_idx[i]
  x <- train[row_idx, feature_cols]
  prob_val <- prob_8[rank_lowest[i]]

  plot_heatmap(
    x,
    main = paste("easiest-8-", i,
                 "\ntrain-row", row_idx,
                 "\nP(class=8)=", round(prob_val, 3))
  )
}

```

```

#-----Q4-----
ks <- 1:30

train_err <- numeric(length(ks))
valid_err <- numeric(length(ks))

for (i in seq_along(ks)){
  k_val <- ks[i]

  knn_train_k <- kknn(
    formula = f_knn,
    train = train,
    test = train,
    k = k_val,
    kernel = "rectangular"
  )

  pred_train_k <- fitted(knn_train_k)
  train_err[i] <- mean(pred_train_k != train$digit)

  knn_valid_k <- kknn(
    formula = f_knn,
    train = train,
    test = valid,
    k = k_val,
    kernel = "rectangular"
  )

  pred_valid_k <- fitted(knn_valid_k)
}

```

```

    valid_err[i] <- mean(pred_valid_k != valid$digit)
  }

cbind(K = ks, train_err, valid_err)

if (!is.null(dev.list())) dev.off()
par(mfrow = c(1,1), mar = c(4,4,2,1))

ylim_range <- range(c(train_err, valid_err))

plot(ks, train_err,
     type = "b", pch = 19,
     col = "blue",
     ylim = ylim_range,
     xlab = "K-(number-of-neighbors)",
     ylab = "Misclassification-error",
     main = "Training-vs-validation-error-for-k-NN")

lines(ks, valid_err,
     type = "b", pch = 19,
     col = "red")

legend("right",
     legend = c("Training-error", "Validation-error"),
     col = c("blue", "red"),
     lty = 1, pch = 19)

#-----Q5-----

ks <- 1:30
ce_valid <- numeric(length(ks))

for (i in seq_along(ks)) {
  k_val <- ks[i]

  # k-NN on validation data
  knn_valid <- kknn(
    formula = f_knn,
    train = train,
    test = valid,
    k = k_val,
    kernel = "rectangular"
  )

  pred_valid <- fitted(knn_valid)

```

```

prob_valid <- knn_valid$prob

y_true      <- valid$digit

class_names <- colnames(prob_valid)
idx_col     <- match(y_true, class_names)

p_true <- prob_valid[cbind(1:nrow(valid), idx_col)]

eps <- 1e-15
ce_valid[i] <- -mean(log(p_true + eps))
}

if (!is.null(dev.list())) dev.off()
par(mfrow = c(1,1), mar = c(4,4,2,1))

plot(ks, ce_valid,
      type = "b", pch = 19,
      xlab = "K-(number-of-neighbors)",
      ylab = "Validation-cross-entropy",
      main = "Validation-cross-entropy-vs-K-for-k-NN")

# If you want the minimum point highlighted:
best_k <- ks[which.min(ce_valid)]
points(best_k, min(ce_valid), pch = 19, col = "red", cex = 1.5)
text(best_k, min(ce_valid),
      labels = paste("Best-K=", best_k),
      pos = 3, col = "red")

```

B Assignment 2 code

```

setwd("~/RStudioProjects/Lab1")
set.seed(12345)
library(caret)

rawData <- read.csv("parkinsons.csv", header = TRUE)
totalRows <- nrow(rawData)
id <- sample(1:totalRows, floor(totalRows * 0.6))
rawTrainData <- rawData[id,]
rawTestData <- rawData[-id,]

scaler <- preProcess(rawTrainData)
initialTrainData <- predict(scaler, rawTrainData)

```

```

initialTestData <- predict(scaler , rawTestData)

LogLikelihood <- function(theta , sigma , y, X) {
  predicted <- X %*% theta
  errors <- y - predicted
  totalLogLike <- sum(dnorm(
    errors ,
    mean = 0 ,
    sd = sigma ,
    log = TRUE
  ))
  return(totalLogLike)
}

Ridge <- function(lambda, theta , sigma , y, X) {
  loglike <- LogLikelihood(theta , sigma , y, X)
  penalty <- lambda * sum(theta^2)
  return(-loglike + penalty)
}

RidgeOpt <- function(lambda, theta , sigma , y, X) {
  costFunction <- function(params) {
    theta_ <- params[1:(length(params) - 1)]
    sigma_ <- exp(params[length(params)])
    penalty <- Ridge(lambda, theta_ , sigma_ , y, X)
    return(penalty)
  }

  return(optim(c(theta , log(sigma)), fn = costFunction))
}

DF <- function(lambda, X) {
  matrixA <- solve((t(X) %*% X) + (lambda * diag(ncol(X))))
  return(sum(diag(X %*% matrixA %*% t(X))))
}

EvaluateRidge <- function(theta , y, testY , X, testX) {
  coefs <- theta$par[1:(length(theta$par) - 1)]

  calcMSE <- function(theta_ , y_ , data_) {
    predictions <- data_ %*% theta_
    residuals <- y_ - predictions
    mse <- mean(residuals^2)
  }
}

```

```

print(paste("Ridge-train-MSE:", calcMSE(coefs, y, X)))
print(paste("Ridge-test-MSE:", calcMSE(coefs, testY, testX)))
}

```

```

LABEL <- "motor_UPDRS"
FORMULA <- motor_UPDRS ~
  0 +
  Jitter... +
  Jitter.Abs. +
  Jitter.RAP +
  Jitter.DDP +
  Jitter.PPQ5 +
  Shimmer +
  Shimmer.dB. +
  Shimmer.APQ3 +
  Shimmer.APQ5 +
  Shimmer.APQ11 +
  Shimmer.DDA +
  NHR +
  HNR +
  RPDE +
  DFA +
  PPE

```

```

trainData <- model.matrix(FORMULA, data = initialTrainData)
trainExpected <- initialTrainData[, LABEL]
testData <- model.matrix(FORMULA, data = initialTestData)
testExpected <- initialTestData[, LABEL]

```

```

model <- lm(FORMULA, data = initialTrainData)
summary(model)

```

```

trainMSE <- mean(residuals(model)^2)
predictions <- predict(model, newdata = as.data.frame(testData))
testMSE <- mean((testExpected - predictions)^2)
print(paste("Train-MSE:", trainMSE))
print(paste("Test-MSE:", testMSE))

```

```

theta <- coef(model)

```

```

print(paste("Log-Likelihood:", LogLikelihood(theta, 1, trainExpected, trainData)

```

```

print(paste(
  "Log-Likelihood + Ridge-Penalty:",
  Ridge(100, theta, 1, trainExpected, trainData)

```

```

))

ridge1 <- RidgeOpt(1, matrix(0, nrow = ncol(trainData)), 1, trainExpected, trainData)
ridge2 <- RidgeOpt(100, matrix(0, nrow = ncol(trainData)), 1, trainExpected, trainData)
ridge3 <- RidgeOpt(1000, matrix(0, nrow = ncol(trainData)), 1, trainExpected, trainData)

EvaluateRidge(ridge1, trainExpected, testExpected, trainData, testData)
EvaluateRidge(ridge2, trainExpected, testExpected, trainData, testData)
EvaluateRidge(ridge3, trainExpected, testExpected, trainData, testData)

print(paste("Degree of freedom (lambda=1)", DF(1, trainData)))
print(paste("Degree of freedom (lambda=100)", DF(100, trainData)))
print(paste("Degree of freedom (lambda=1000)", DF(1000, trainData)))

```

C Assignment 3 code

```

setwd(getwd())
list.files()
data <- read.csv("tdde01/3/pima-indians-diabetes.csv", header= TRUE)

colnames(data) <- c("times_pregnant", "glucose", "blood_pressure", "skinfold", "diabetes")

glucose <- data$glucose
age <- data$age
diabetes <- data$diabetes

plot(age, glucose, col=factor(diabetes))

glmobj <- glm(formula = diabetes ~ glucose + age, data = data, family = binomial)
summary(glmobj)
coef(glmobj)
result <- predict(glmobj, type = "response")

pred <- ifelse(result > 0.5, 1, 0)
plot(age, glucose, col=factor(pred))
mean(pred != data$diabetes)
lines(age_vals, glucose_boundary, col="black")

pred <- ifelse(result > 0.2, 1, 0)
plot(age, glucose, col=factor(pred))
lines(age_vals, glucose_boundary, col="blue")

pred <- ifelse(result > 0.8, 1, 0)
plot(age, glucose, col=factor(pred))

```

```

lines(age_vals , glucose_boundary , col="blue")

mean(pred != data$diabetes)

coefficients <- coef(glmobj)
intercept <- coefficients[1]
glucoseCoefficient <- coefficients[2]
ageCoefficient <- coefficients[3]

age_vals <- seq(min(data$age), max(data$age), length.out = 10)

glucose_boundary <- -(intercept + ageCoefficient * age_vals) / glucoseCoefficient

z1 <- data$times_pregnant ** 4
z2 <- data$times_pregnant ** 3 * data$glucose
z3 <- data$times_pregnant ** 2 * data$glucose ** 2
z4 <- data$times_pregnant * data$glucose ** 3
z5 <- data$glucose ** 4

glmobj <- glm(formula = diabetes ~ glucose + age + z1 + z2 + z3 + z4 + z5, data = data)
summary(glmobj)
result <- predict(glmobj, type = "response")
pred <- ifelse(result > 0.5, 1, 0)

plot(age, glucose , col = factor(pred))
mean(pred != data$diabetes)

```